Behavior trees typically use a pattern called the *blackboard* to communicate across nodes. Blackboards, like those in `py_trees`, use a key/value store to save information and make it accessible across nodes. It also provides for several controls, such as limiting access to specific nodes.

We deferred to using files for communication because of their simplicity and transparency. At some point, agentic systems are expected to consume much more information and in different formats than those designed for blackboards. Blackboards must either become more sophisticated or be integrated with file storage solutions.

Let's build an ABT using back chaining. We could tackle a variety of goals, but one interesting and perhaps meta goal is to build an ABT that helps build assistants. So let's first present our goal as a statement "Create an assistant that can help me do {task}":

- *Required actions*: (working backwards)
  - Create an assistant.
  - Verify the assistant.
  - Test the assistant.
  - Name the assistant.
  - Give the assistant the relevant instructions.
- *Identified condition:*
  - Verify the assistant.
- *Determine communication patterns*: To keep things interesting, we'll run all assistants on the same message thread.
- *Construct the tree*: To construct the tree, let's first reverse the order of actions and mark each of the element's actions and conditions accordingly:
  - (action) Give the assistant relevant instructions to help a user with a given task.
  - (action) Name the assistant.
  - (action) Test the assistant.
  - (condition) Verify the assistant.
  - (action) Create the assistant.